MANIPAL UNIVERSITY JAIPUR
University under Section 2(f) of the UGC Act

Vehicle Rental Management System: A Relational Database Design with MySQL

Project Report

Submitted by:

Ritvik Jindal
Reg No: 2502051953
Programme: B.Tech CSE Section: N3

Submitted to:

Dr. Stuti Pandey

Department of Computer Science and Engineering
Manipal University Jaipur
Dehmi Kalan, Near GVK Toll Plaza, Jaipur-Ajmer Expressway,
Jaipur, Rajasthan 303007

Abstract

The Vehicle Rental Management System is a web application for managing vehicle rentals, and this report documents the relational database on which it depends. The objective of the project was to design a normalised MySQL database that stores customers and administrators, vehicles and their images, promotional codes, rental bookings, payments and maintenance records while preserving data integrity. The methodology involved entity-relationship modelling, mapping the model to seven relational tables, and defining primary keys, foreign keys, unique constraints, enumerated types, default values, check constraints and twelve secondary indexes in a MySQL schema script. The design places each entity in its own table, retains the price breakdown of every rental as a historical record, and uses restrictive referential actions to protect rental and payment history. SQL queries for retrieval, aggregation and reporting, together with a transaction pattern for booking and payment, show how the schema supports the main operations of a rental business. The report concludes that a carefully constrained relational schema is a dependable foundation for a rental platform, and it identifies business rules, notably the prevention of overlapping bookings, that should be strengthened in future work.

Keywords: RDBMS, MySQL, ER diagram, relational schema, normalisation, integrity constraints, indexing, transactions.

Contents

Abstract 1

Frontend Interface Screenshots 3

1 Introduction 4

1.1 Project Overview 4

1.2 Objectives of the Project 4

1.3 Importance and Relevance 5

1.4 Scope and Limitations 5

2 System Design 6

2.1 Database Design Overview 6

2.2 Entity-Relationship (ER) Diagram 6

2.3 Relational Schema 7

2.4 Description of Tables and Attributes 8

2.5 Keys and Integrity Constraints 12

2.6 Normalisation 13

2.7 Indexes 14

3 Database Implementation 15

3.1 Database Environment and DDL 15

3.2–3.4 DML and Retrieval Queries 16

3.5 Transactions and ACID Properties 19

3.6 Referential Actions 20

3.7 Constraint Verification 20

4 Conclusion 22

5 Future Work 23

References 24

Frontend Interface Screenshots

Vehicle Rental Management System

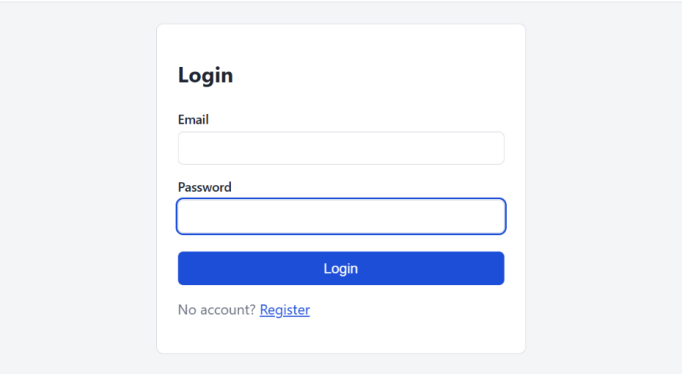


Figure 1: Login page

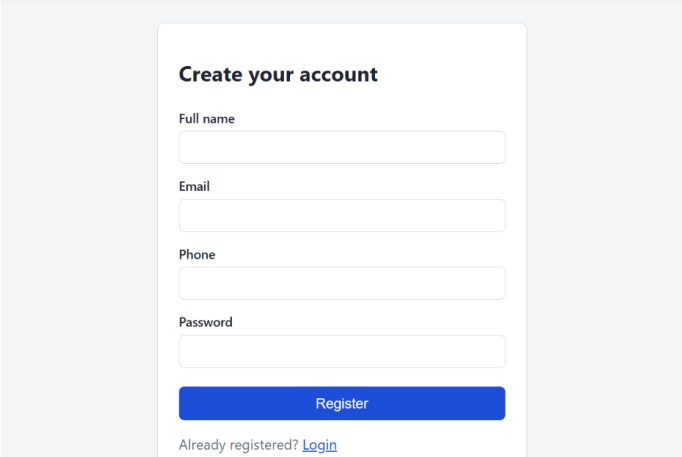


Figure 2: Registration page

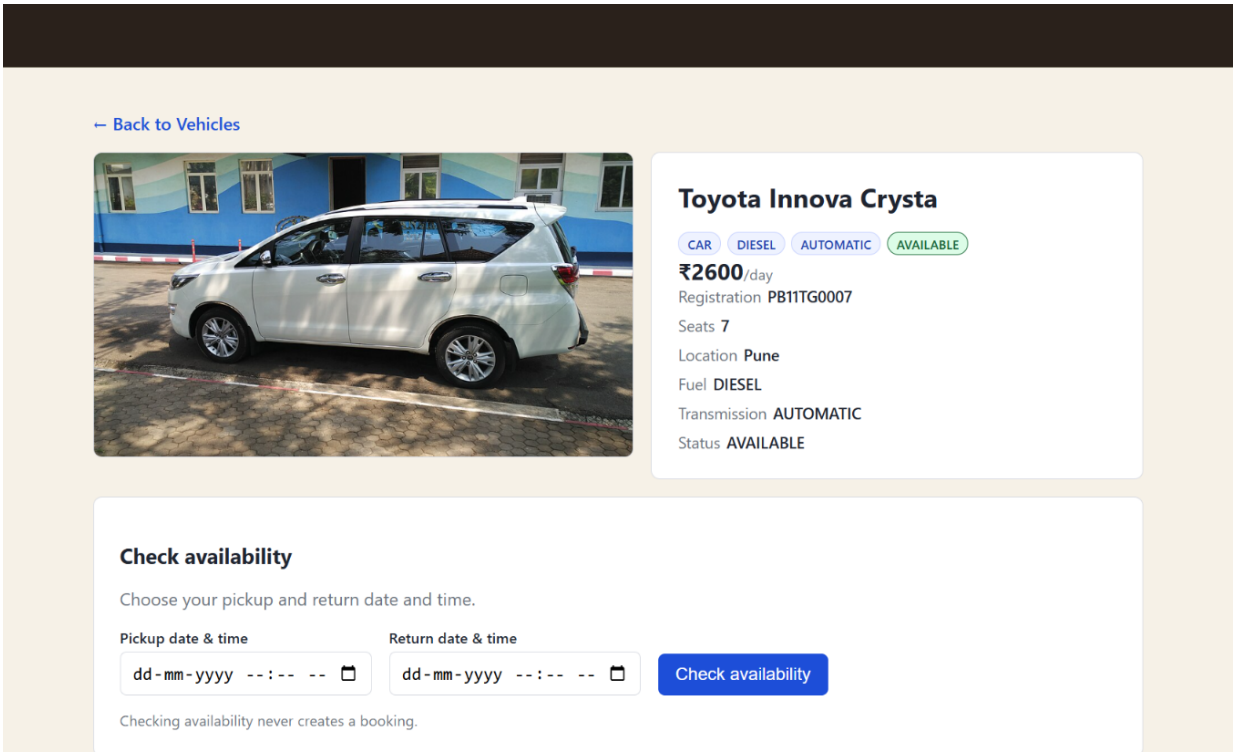


Figure 3: Vehicle details and availability

Screenshots show the frontend interface; they do not independently verify backend workflows.

Chapter 1

Introduction

1.1 Project Overview

Vehicle rental businesses manage several kinds of connected information: customers who book vehicles, the vehicles themselves together with their images and maintenance histories, promotional offers, rental periods with their price calculations, and online payments. When these facts are kept in separate files or in a single wide table, values are duplicated, updates become inconsistent and orphan records appear. The Vehicle Rental Management System addresses this problem by placing all of this data in a relational database. The system is a web application with a React frontend and a Java-based backend, and this report concentrates on the MySQL database that stores and protects its data.

The database consists of seven tables: `users`, `vehicles`, `vehicle_images`, `promo_codes`, `rentals`, `payments` and `maintenance`. Primary and foreign keys connect the tables, and constraints enforce the validity rules the business requires, such as unique registration numbers, non-negative amounts and return times that fall after pickup times.

1.2 Objectives of the Project

The primary objectives of this project were as follows:

- To model the entities and relationships involved in vehicle rental operations using an entity-relationship diagram.
- To map the model to a relational schema in MySQL with appropriate data types, primary keys and foreign keys.
- To enforce data integrity through unique, default, enumerated and check constraints.
- To reduce avoidable redundancy by separating related entities into their own tables, and to document any deliberate exceptions.
- To provide SQL statements for common retrieval, aggregation and reporting tasks.
- To define indexes on the attributes most often used for filtering and joining.
- To describe transaction handling for operations that span several tables, such as a booking together with its payment.

1.3 Importance and Relevance

A rental platform handles money, schedules and physical assets at the same time. A payment must always belong to a real rental, a rental must always belong to a real customer and vehicle, and the price agreed at booking time must remain available even after the vehicle's current rate changes. These guarantees are best provided by the database itself rather than left entirely to application code. The project therefore demonstrates how keys, constraints, normalisation, indexes and transactions work together in a realistic business domain, and the resulting design can serve as a template for other booking-oriented systems.

1.4 Scope and Limitations

The scope of this report is the relational database layer. It covers the entities and relationships, the relational schema and data dictionary, integrity constraints, normalisation, indexes, representative SQL statements and the use of transactions. The design of the user interface and of the Java service classes is outside the scope of the report.

Limitations of the current database design include:

- The schema does not itself prevent two bookings of the same vehicle from overlapping in time; this rule must be enforced by application logic or by additional database code (Section 3.5).
- `payments.user_id` and `rentals.user_id` are stored independently, so the schema does not guarantee that a payment's user is the user who owns the rental.
- The rules that `used_count` must not exceed `usage_limit` and that `maximum_discount` must be non-negative are not expressed as constraints.
- Payment records store Razorpay identifiers only; communication with the payment gateway is handled outside the database and is not covered here.
- Refunds are represented only by the `REFUNDED` payment status; partial refunds and refund amounts are not modelled.

Chapter 2

System Design

2.1 Database Design Overview

The heart of the Vehicle Rental Management System is its MySQL database. The design follows the relational model: each table represents one entity, a primary key identifies every row, and foreign keys connect rows across tables so that, for example, a rental cannot exist without a valid user and a valid vehicle. Descriptive data is stored with the entity it describes. Vehicle images, for instance, are kept in their own table so that a vehicle can have any number of images without repeating its other attributes. The schema is created in MySQL 8 and relies on the InnoDB storage engine, which supports foreign keys and transactions [3].

The `users` table holds both customers and administrators, distinguished by a role column. The `rentals` table is the central table of the design: it links a user to a vehicle for a period of time, records the price calculation, optionally refers to a promotional code, and is the parent of the `payments` table.

2.2 Entity-Relationship (ER) Diagram

The ER diagram in Figure 2.1 provides a visual blueprint of the database. Each box is an entity with its attributes; primary keys are underlined, foreign keys are in italics, and the labels **1** and **N** give the cardinality at each end of a relationship.

The key entities and relationships include:

- **Entities:** User, Vehicle, VehicleImage, PromoCode, Rental, Payment and Maintenance.
- **One-to-Many Relationships:** A `User` can make many `Rentals` and many `Payments`. A `Vehicle` can appear in many `Rentals` over time and can have many `VehicleImages` and many `Maintenance` records. A `Rental` can have many `Payments`. A `PromoCode` can be used in many `Rentals`.
- **Many-to-Many Relationship:** Users and vehicles are related many-to-many, because a user can rent many vehicles and a vehicle can be rented by many users. The `Rental` entity resolves this relationship and also carries the booking attributes (period, price breakdown and status).
- **Optional Relationship:** `rentals.promo_id` allows `NULL`, since a rental may have no promotional code.

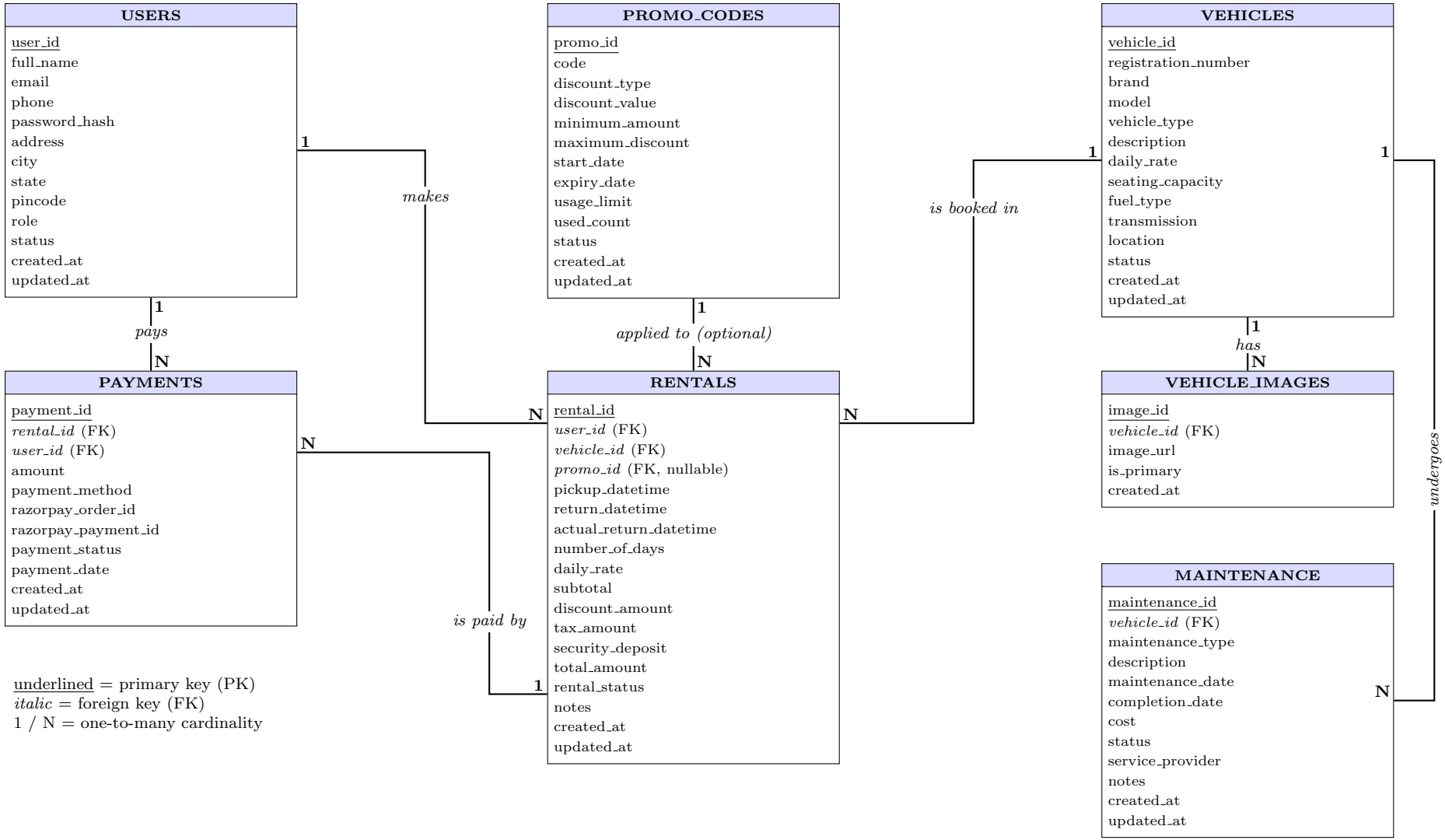


Figure 2.1: ER Diagram for the Vehicle Rental Management System

2.3 Relational Schema

The ER model is mapped to the following relational schema. Primary keys are underlined and foreign keys are shown in italics.

- USERS(user_id, full_name, email, phone, password_hash, address, city, state, pincode, role, status, created_at, updated_at)
- VEHICLES(vehicle_id, registration_number, brand, model, vehicle_type, description, daily_rate, seating_capacity, fuel_type, transmission, location, status, created_at, updated_at)
- VEHICLE_IMAGES(image_id, *vehicle_id*, image_url, is_primary, created_at)
- PROMO_CODES(promo_id, code, discount_type, discount_value, minimum_amount, maximum_discount, start_date, expiry_date, usage_limit, used_count, status, created_at, updated_at)
- RENTALS(rental_id, *user_id*, *vehicle_id*, *promo_id*, pickup_datetime, return_datetime, actual_return_datetime, number_of_days, daily_rate, subtotal, discount_amount, tax_amount, security_deposit, total_amount, rental_status, notes, created_at, updated_at)
- PAYMENTS(payment_id, *rental_id*, *user_id*, amount, payment_method, razorpay_order_id, razorpay_payment_id, payment_status, payment_date, created_at, updated_at)
- MAINTENANCE(maintenance_id, *vehicle_id*, maintenance_type, description, maintenance_date, completion_date, cost, status, service_provider, notes, created_at, updated_at)

Table 2.1 lists the seven foreign-key relationships together with their cardinality and referential actions.

Table 2.1: Foreign-Key Relationships and Referential Actions

Parent	Child.Column	Cardinality	ON DELETE	ON UPDATE
users	rentals.user_id	1:N	RESTRICT	CASCADE
vehicles	rentals.vehicle_id	1:N	RESTRICT	CASCADE
promo_codes	rentals.promo_id	1:N (optional)	SET NULL	CASCADE
rentals	payments.rental_id	1:N	RESTRICT	CASCADE
users	payments.user_id	1:N	RESTRICT	CASCADE
vehicles	vehicle_images.vehicle_id	1:N	RESTRICT	CASCADE
vehicles	maintenance.vehicle_id	1:N	RESTRICT	CASCADE

2.4 Description of Tables and Attributes

The schema is composed of the following seven tables, defined in the project's `schema.sql` script. In the Key column, PK denotes a primary key, UNI a unique column and FK a

foreign key. Unless stated otherwise, a column is `NOT NULL`. `created_at` and `updated_at` are timestamps: the first defaults to the time of insertion and the second is refreshed automatically whenever the row changes.

Table 2.2: Users Table Schema

Field	Type	Key	Description
`user_id`	BIGINT	PK	Unique identifier; auto-increment.
`full_name`	VARCHAR(100)		User's full name.
`email`	VARCHAR(150)	UNI	Email address.
`phone`	VARCHAR(20)	UNI	Phone number.
`password_hash`	VARCHAR(255)		Hashed password; no plain-text password is stored.
`address`	VARCHAR(255)		Street address; optional.
`city`	VARCHAR(100)		City; optional.
`state`	VARCHAR(100)		State; optional.
`pincode`	VARCHAR(10)		Postal code stored as text; optional.
`role`	ENUM		`CUSTOMER` or `ADMIN`; default `CUSTOMER`.
`status`	ENUM		`ACTIVE` or `INACTIVE`; default `ACTIVE`.
`created_at`	DATETIME		Creation timestamp.
`updated_at`	DATETIME		Last-update timestamp.

Table 2.3: Vehicles Table Schema

Field	Type	Key	Description
`vehicle_id`	BIGINT	PK	Unique identifier; auto-increment.
`registration_number`	VARCHAR(30)	UNI	Vehicle registration number.
`brand`	VARCHAR(80)		Manufacturer or brand.
`model`	VARCHAR(100)		Vehicle model.
`vehicle_type`	ENUM		`CAR`, `BIKE`, `BUS` or `TRUCK`.
`description`	TEXT		Free-text description; optional.
`daily_rate`	DECIMAL(10,2)		Current rental rate per day; must be ≥ 0.
`seating_capacity`	INT		Number of seats; must be > 0.
`fuel_type`	ENUM		`PETROL`, `DIESEL`, `ELECTRIC`, `CNG` or `HYBRID`.
`transmission`	ENUM		`MANUAL`, `AUTOMATIC` or `NOT_APPLICABLE`; nullable, default `NOT_APPLICABLE`.
`location`	VARCHAR(150)		Location where the vehicle is available.
`status`	ENUM		`AVAILABLE`, `RENTED`, `MAINTENANCE` or `INACTIVE`; default `AVAILABLE`.
`created_at`	DATETIME		Creation timestamp.
`updated_at`	DATETIME		Last-update timestamp.

Table 2.4: Vehicle Images Table Schema

Field	Type	Key	Description
`image_id`	BIGINT	PK	Unique identifier; auto-increment.
`vehicle_id`	BIGINT	FK	References `vehicles(vehicle_id)`.
`image_url`	VARCHAR(500)		Location (URL) of the image.
`is_primary`	BOOLEAN		Marks the primary image; default `FALSE`.
`created_at`	DATETIME		Creation timestamp.

Table 2.5: Promo Codes Table Schema

Field	Type	Key	Description
`promo_id`	BIGINT	PK	Unique identifier; auto-increment.
`code`	VARCHAR(50)	UNI	Promotional code entered by the customer.
`discount_type`	ENUM		`PERCENTAGE` or `FIXED`.
`discount_value`	DECIMAL(10,2)		Discount amount or percentage; must be ≥ 0.
`minimum_amount`	DECIMAL(10,2)		Minimum booking amount for the code; default 0, must be ≥ 0.
`maximum_discount`	DECIMAL(10,2)		Upper limit on the discount; optional.
`start_date`	DATE		First valid date.
`expiry_date`	DATE		Last valid date; must not precede `start_date`.
`usage_limit`	INT		Maximum number of uses; optional, must be > 0 if present.
`used_count`	INT		Number of times used; default 0.
`status`	ENUM		`ACTIVE` or `INACTIVE`; default `ACTIVE`.
`created_at`	DATETIME		Creation timestamp.
`updated_at`	DATETIME		Last-update timestamp.

Table 2.6: Rentals Table Schema

Field	Type	Key	Description
`rental_id`	BIGINT	PK	Unique identifier; auto-increment.
`user_id`	BIGINT	FK	References `users(user_id)`.
`vehicle_id`	BIGINT	FK	References `vehicles(vehicle_id)`.
`promo_id`	BIGINT	FK	References `promo_codes(promo_id)`; nullable.
`pickup_datetime`	DATETIME		Planned pickup date and time.
`return_datetime`	DATETIME		Planned return; must be later than pickup.

Field	Type	Key	Description
actual_return_datetime	DATETIME		Actual return time; nullable until the vehicle is returned.
number_of_days	INT		Number of rental days; must be > 0.
daily_rate	DECIMAL(10,2)		Daily rate agreed at booking time; must be ≥ 0.
subtotal	DECIMAL(10,2)		Price before discount and tax; must be ≥ 0.
discount_amount	DECIMAL(10,2)		Discount applied; default 0, must be ≥ 0.
tax_amount	DECIMAL(10,2)		Tax charged; default 0, must be ≥ 0.
security_deposit	DECIMAL(10,2)		Deposit collected; default 0, must be ≥ 0.
total_amount	DECIMAL(10,2)		Total payable; must be ≥ 0.
rental_status	ENUM		PENDING, CONFIRMED, ACTIVE, COMPLETED or CANCELLED; default PENDING.
notes	TEXT		Booking notes; optional.
created_at	DATETIME		Creation timestamp.
updated_at	DATETIME		Last-update timestamp.

Table 2.7: Payments Table Schema

Field	Type	Key	Description
payment_id	BIGINT	PK	Unique identifier; auto-increment.
rental_id	BIGINT	FK	References rentals(rental_id).
user_id	BIGINT	FK	References users(user_id).
amount	DECIMAL(10,2)		Payment amount; must be > 0.
payment_method	ENUM		Currently only RAZORPAY; default RAZORPAY.
razorpay_order_id	VARCHAR(100)	UNI	Provider order identifier; nullable.
razorpay_payment_id	VARCHAR(100)	UNI	Provider payment identifier; nullable.
payment_status	ENUM		PENDING, SUCCESS, FAILED, CANCELLED or REFUNDED; default PENDING.
payment_date	DATETIME		Date and time of payment; nullable.
created_at	DATETIME		Creation timestamp.
updated_at	DATETIME		Last-update timestamp.

Table 2.8: Maintenance Table Schema

Field	Type	Key	Description
maintenance_id	BIGINT	PK	Unique identifier; auto-increment.

Field	Type	Key	Description
vehicle_id	BIGINT	FK	References vehicles(vehicle_id).
maintenance_type	VARCHAR(100)		Type of maintenance work.
description	TEXT		Description of the work; optional.
maintenance_date	DATE		Date the maintenance is scheduled or starts.
completion_date	DATE		Completion date; nullable, not earlier than maintenance_date.
cost	DECIMAL(10,2)		Cost of the work; default 0, must be ≥ 0.
status	ENUM		SCHEDULED, IN_PROGRESS, COMPLETED or CANCELLED; default SCHEDULED.
service_provider	VARCHAR(150)		Workshop or provider; optional.
notes	TEXT		Additional notes; optional.
created_at	DATETIME		Creation timestamp.
updated_at	DATETIME		Last-update timestamp.

2.5 Keys and Integrity Constraints

2.5.1 Primary Keys

Every table has a primary key consisting of an auto-incrementing BIGINT identifier: user_id, vehicle_id, image_id, promo_id, rental_id, payment_id and maintenance_id. Primary keys identify rows uniquely and cannot be NULL.

2.5.2 Unique Constraints

Unique constraints are defined on users.email, users.phone, vehicles.registration_number, promo_codes.code, payments.razorpay_order_id and payments.razorpay_payment_id. They prevent duplicate accounts, duplicate vehicles, duplicate promotional codes and duplicate payment-provider identifiers. In MySQL a unique index permits multiple NULL values, so payment records without a provider identifier do not conflict with one another.

2.5.3 Foreign Keys

Seven foreign keys connect the tables (Table 2.1). Six use ON DELETE RESTRICT, which prevents a parent row from being deleted while dependent rows exist, so a vehicle with rental history cannot be removed accidentally. The foreign key from rentals.promo_id uses ON DELETE SET NULL, so deleting a promotional code leaves the rental in place with a NULL promo reference. All seven use ON UPDATE CASCADE, which propagates a change of a parent key to the child rows.

2.5.4 Check Constraints

Check constraints express basic validity rules for amounts, counts and dates. The schema defines seventeen named checks, summarised in Table 2.9. MySQL enforces `CHECK` constraints from version 8.0.16 onwards [2].

Table 2.9: Check Constraints in the Schema

Table	Constraint	Rule
vehicles	`chk_daily_rate`	daily_rate ≥ 0
vehicles	`chk_seating_capacity`	seating_capacity > 0
promo_codes	`chk_discount_value`	discount_value ≥ 0
promo_codes	`chk_minimum_amount`	minimum_amount ≥ 0
promo_codes	`chk_usage_limit`	usage_limit is `NULL` or > 0
promo_codes	`chk_expiry_date`	expiry_date $\geq$ start_date
rentals	`chk_return_datetime`	return_datetime $>$ pickup_datetime
rentals	`chk_number_of_days`	number_of_days > 0
rentals	`chk_rental_daily_rate`	daily_rate ≥ 0
rentals	`chk_subtotal`	subtotal ≥ 0
rentals	`chk_discount_amount`	discount_amount ≥ 0
rentals	`chk_tax_amount`	tax_amount ≥ 0
rentals	`chk_security_deposit`	security_deposit ≥ 0
rentals	`chk_total_amount`	total_amount ≥ 0
payments	`chk_payment_amount`	amount > 0
maintenance	`chk_maintenance_cost`	cost ≥ 0
maintenance	`chk_completion_date`	completion_date is `NULL` or $\geq$ maintenance_date

2.5.5 Domain Constraints

Enumerated columns (`role`, `vehicle_type`, `fuel_type`, `rental_status`, `payment_status` and others) restrict a column to a fixed set of values, `NOT NULL` makes a value mandatory, and default values supply common states such as `PENDING` for a new rental or `AVAILABLE` for a new vehicle.

2.6 Normalisation

Normalisation structures relations so that each fact is stored once, which reduces redundancy and update anomalies [5, 6]. Table 2.10 lists the candidate keys and the dependency of the remaining attributes for each table.

2.6.1 First Normal Form (1NF)

All attributes hold single, atomic values and there are no repeating groups. Instead of storing a list of image links in a vehicle row, each image is a separate row in `vehicle_images`. Rentals, payments and maintenance records are likewise stored as separate rows rather than as multi-valued columns.

Table 2.10: Candidate Keys and Functional Dependencies

Table	Candidate keys	Functional dependencies
users	user_id, email, phone	Each candidate key determines all other attributes.
vehicles	vehicle_id, registration_number	Each candidate key determines all other attributes.
vehicle_images	image_id	image_id → all other attributes.
promo_codes	promo_id, code	Each candidate key determines all other attributes.
rentals	rental_id	rental_id → all other attributes.
payments	payment_id (also the provider identifiers when present)	payment_id → all other attributes.
maintenance	maintenance_id	maintenance_id → all other attributes.

2.6.2 Second Normal Form (2NF)

Every table has a single-column primary key, so no non-key attribute can depend on only part of a key. All tables that are in 1NF are therefore also in 2NF. Descriptive data is held with its own entity: vehicle characteristics in `vehicles`, contact details in `users` and work details in `maintenance`.

2.6.3 Third Normal Form (3NF)

A relation is in 3NF when no non-key attribute depends on the key through another non-key attribute. Brand and model are stored once in `vehicles` and are not copied into each rental; user contact data is stored once in `users`; promotion rules are stored once in `promo_codes`. Table 2.11 summarises the result for each table, including three deliberate exceptions.

2.6.4 Benefits of Normalisation

- A user can change contact details without touching any rental record.
- Vehicle information is maintained in one place while rental history remains intact.
- Several images, payments and maintenance records can be attached to one vehicle or rental without duplicating descriptive data.
- Promotion rules can be changed once without editing every rental that used them.

2.7 Indexes

An index is an auxiliary structure that lets MySQL find rows without scanning a whole table. Primary keys and unique constraints are indexed automatically. In addition, the schema script creates twelve secondary indexes (Table 2.12) on columns that are frequently used in filters and joins. Each index is created only if an index of the same name

Table 2.11: Normal Form Achieved by Each Table

Table	Normal form	Notes and exceptions
users	3NF*	*If a postal code is taken to determine city and state, `pincode` → `city`, `state` is a transitive dependency. The three columns are kept together because addresses are entered by users and are optional.
vehicles	3NF	`status` records the current operational state. The value `RENTED` can also be derived from `rentals`; it is retained for fast availability lookups.
vehicle_images	3NF	No exceptions.
promo_codes	3NF	No exceptions.
rentals	3NF*	*`number_of_days`, `daily_rate`, `subtotal` and `total_amount` are stored as a snapshot of the booking. They are not recomputed from the vehicle's current rate, which may change after the booking.
payments	3NF*	*`user_id` can normally be derived from the rental. It is stored so that payments can be queried directly by user.
maintenance	3NF	No exceptions.

does not already exist, by checking `information_schema.statistics` before executing the `CREATE INDEX` statement, so the script can be run again safely against an existing database. The foreign-key columns `rentals.promo_id` and `vehicle_images.vehicle_id` have no explicit index; InnoDB creates an index for them automatically because a foreign key requires one [3].

Indexes use storage and add work to every insert and update, so they should be chosen to match real query patterns. The `EXPLAIN` statement can be used to confirm that a query uses an intended index [4].

Chapter 3

Database Implementation

3.1 Database Environment

The database is named `vehicle_rental_db` and is defined in MySQL by the `schema.sql` script. The script uses the MySQL types `BIGINT`, `VARCHAR`, `DECIMAL`, `DATETIME`, `DATE`, `TEXT`, `BOOLEAN` and `ENUM`. Every table is created with `CREATE TABLE IF NOT EXISTS`, so running the script against a database that already contains the tables does not fail. The script requires MySQL 8.0.16 or later, the first release that enforces `CHECK` constraints [2]. The statements in this chapter can be run in MySQL Workbench or any other MySQL client.

3.2 SQL Data Definition Language (DDL)

Data Definition Language statements create and change the structure of the database. Listing 3.1 shows the definition of the `vehicles` table, which illustrates primary keys, unique and `NOT NULL` columns, enumerated types, defaults and named check constraints.

```sql
CREATE TABLE IF NOT EXISTS vehicles (
    vehicle_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    registration_number VARCHAR(30) NOT NULL UNIQUE,
    brand VARCHAR(80) NOT NULL,
    model VARCHAR(100) NOT NULL,
    vehicle_type ENUM('CAR','BIKE','BUS','TRUCK') NOT NULL,
    description TEXT,
    daily_rate DECIMAL(10,2) NOT NULL,
    seating_capacity INT NOT NULL,
    fuel_type ENUM('PETROL','DIESEL','ELECTRIC','CNG','HYBRID') NOT NULL,
    transmission ENUM('MANUAL','AUTOMATIC','NOT_APPLICABLE')
        DEFAULT 'NOT_APPLICABLE',
    location VARCHAR(150) NOT NULL,
    status ENUM('AVAILABLE','RENTED','MAINTENANCE','INACTIVE')
        NOT NULL DEFAULT 'AVAILABLE',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT chk_daily_rate CHECK (daily_rate >= 0),
    CONSTRAINT chk_seating_capacity CHECK (seating_capacity > 0)
);
```

Listing 3.1: DDL for the vehicles table

The secondary indexes are created by statements that first check the metadata, as shown in Listing 3.2.

```
SET @sql := IF(NOT EXISTS (
    SELECT 1 FROM information_schema.statistics
    WHERE table_schema = DATABASE()
      AND table_name = 'vehicles'
      AND index_name = 'idx_vehicle_status'),
  'CREATE INDEX idx_vehicle_status ON vehicles(status)',
  'DO 0');
PREPARE stmt FROM @sql;
EXECUTE stmt;
DEALLOCATE PREPARE stmt;
```

Listing 3.2: Idempotent creation of an index

3.3 SQL Data Manipulation Language (DML)

Data Manipulation Language statements insert, change and remove rows. The statements below use example values; the identifiers must correspond to existing rows when they are executed.

```
INSERT INTO vehicles
  (registration_number, brand, model, vehicle_type,
   daily_rate, seating_capacity, fuel_type, transmission,
   location, status)
VALUES
  ('RJ14AB1234', 'Maruti', 'Swift', 'CAR',
   1800.00, 5, 'PETROL', 'MANUAL', 'Jaipur', 'AVAILABLE');
```

Listing 3.3: Inserting a vehicle

```
-- Mark a vehicle as under maintenance
UPDATE vehicles
SET status = 'MAINTENANCE'
WHERE vehicle_id = 1;

-- Remove an image that is no longer needed
DELETE FROM vehicle_images
WHERE image_id = 1;
```

Listing 3.4: Updating and deleting rows

A `DELETE` on a parent row such as a vehicle that still has rentals is rejected by the foreign-key rules (Section 3.6); an inactive status is used when history must be preserved.

3.4 SQL Retrieval Queries

The following queries show typical retrieval and reporting operations. They only read data and do not change the database.

List Available Vehicles

```sql
SELECT vehicle_id, brand, model, vehicle_type,
       daily_rate, location
FROM vehicles
WHERE status = 'AVAILABLE'
ORDER BY daily_rate ASC;
```

Listing 3.5: Filtering and sorting vehicles

The query filters vehicles by status and lists them from the lowest to the highest daily rate. It can use `idx_vehicle_status`.

Find Rentals of a User

```sql
SELECT u.full_name, r.rental_id,
       r.pickup_datetime, r.return_datetime,
       r.total_amount, r.rental_status
FROM users AS u
JOIN rentals AS r ON r.user_id = u.user_id
WHERE u.user_id = 1
ORDER BY r.created_at DESC;
```

Listing 3.6: Joining users and rentals

The inner join matches every rental with the user referenced by its foreign key.

View Payments of a Rental

```sql
SELECT r.rental_id, p.payment_id, p.amount,
       p.payment_method, p.payment_status, p.payment_date
FROM rentals AS r
JOIN payments AS p ON p.rental_id = r.rental_id
WHERE r.rental_id = 1
ORDER BY p.created_at DESC;
```

Listing 3.7: Joining rentals and payments

Count Rentals by Status

```sql
SELECT rental_status, COUNT(*) AS rental_count
FROM rentals
GROUP BY rental_status
ORDER BY rental_count DESC;
```

Listing 3.8: Aggregation with GROUP BY

Total of Successful Payments per User

```sql
SELECT user_id, SUM(amount) AS successful_total
FROM payments
WHERE payment_status = 'SUCCESS'
GROUP BY user_id
ORDER BY successful_total DESC;
```

Listing 3.9: SUM with a status filter

The result is a report of the payment rows recorded as successful in the database.

Maintenance History of a Vehicle

```sql
SELECT v.registration_number, v.brand, v.model,
       m.maintenance_type, m.maintenance_date,
       m.completion_date, m.cost, m.status
FROM vehicles AS v
JOIN maintenance AS m ON m.vehicle_id = v.vehicle_id
WHERE v.vehicle_id = 1
ORDER BY m.maintenance_date DESC;
```

Listing 3.10: Joining vehicles and maintenance

Rentals with Optional Promo Codes

```sql
SELECT r.rental_id, r.total_amount,
       pc.code, pc.discount_type, r.discount_amount
FROM rentals AS r
LEFT JOIN promo_codes AS pc ON pc.promo_id = r.promo_id
ORDER BY r.created_at DESC;
```

Listing 3.11: LEFT JOIN for an optional relationship

A `LEFT JOIN` is used because `rentals.promo_id` is nullable: rentals without a promo code stay in the result with `NULL` promo columns.

Vehicles That Have Never Been Rented

```sql
SELECT vehicle_id, registration_number, brand, model
FROM vehicles
WHERE vehicle_id NOT IN (SELECT vehicle_id FROM rentals);
```

Listing 3.12: Subquery with NOT IN

Customers with Repeat Completed Rentals

```sql
SELECT u.user_id, u.full_name, COUNT(*) AS completed_rentals
FROM users AS u
JOIN rentals AS r ON r.user_id = u.user_id
WHERE r.rental_status = 'COMPLETED'
GROUP BY u.user_id, u.full_name
HAVING COUNT(*) >= 2
ORDER BY completed_rentals DESC;
```

Listing 3.13: GROUP BY with HAVING

3.4.1 Query Concepts Used

The queries demonstrate `WHERE` for filtering, `ORDER BY` for sorting, `INNER JOIN` and `LEFT JOIN` for combining related tables, `COUNT` and `SUM` for aggregation, `GROUP BY` and `HAVING` for grouped summaries, and a subquery with `NOT IN`.

3.5 Transactions and ACID Properties

A transaction groups several database operations into one logical unit. Creating a rental and recording its payment is a typical example, because either both records must exist or neither should. Transactions provide the ACID properties:

- **Atomicity:** all statements of the transaction take effect, or none of them do.
- **Consistency:** integrity constraints hold before and after the transaction.
- **Isolation:** concurrent transactions do not interfere in ways that break correctness.
- **Durability:** committed changes persist.

Listing 3.14 shows a booking transaction. The vehicle row is locked first, so that concurrent bookings of the same vehicle are processed one after another by InnoDB. The overlap check is then executed, and the rental and payment are inserted only if no overlapping booking is found. The monetary values and identifiers are examples, and the totals are calculated by the application.

```
START TRANSACTION;

-- 1. Lock the vehicle row so concurrent bookings are serialised
SELECT vehicle_id FROM vehicles
WHERE vehicle_id = 1 AND status <> 'INACTIVE'
FOR UPDATE;

-- 2. Check for overlapping bookings (continue only if the count is 0)
SELECT COUNT(*) AS overlapping
FROM rentals
WHERE vehicle_id = 1
  AND rental_status IN ('PENDING','CONFIRMED','ACTIVE')
  AND pickup_datetime < '2026-11-05 10:00:00'
  AND return_datetime > '2026-11-01 10:00:00';

-- 3. Create the rental
INSERT INTO rentals
  (user_id, vehicle_id, pickup_datetime, return_datetime,
   number_of_days, daily_rate, subtotal, discount_amount,
   tax_amount, security_deposit, total_amount, rental_status)
VALUES
  (1, 1, '2026-11-01 10:00:00', '2026-11-05 10:00:00',
   4, 1800.00, 7200.00, 0, 360.00, 2000.00, 9560.00, 'PENDING');

-- 4. Record the payment for the new rental
INSERT INTO payments
  (rental_id, user_id, amount, razorpay_order_id, payment_status)
VALUES
  (LAST_INSERT_ID(), 1, 9560.00, 'order_EXAMPLE001', 'PENDING');

COMMIT;   -- or ROLLBACK if any step fails
```

Listing 3.14: Booking transaction for a rental and its payment

Foreign keys and check constraints preserve consistency, but they cannot by themselves stop two customers from booking the same vehicle for overlapping periods. The locking and overlap check shown above, or an equivalent trigger or stored procedure, is needed for that rule (see Chapter 5).

3.6 Referential Actions

The schema uses different delete behaviours according to the relationship. `RESTRICT` prevents deletion of a parent row while child rows exist: a vehicle that has images, rentals or maintenance records cannot be deleted, and a rental that has payments cannot be deleted. This protects rental, payment and maintenance history. `ON UPDATE CASCADE` carries a change of a parent key to all child rows. `SET NULL` on the promotional-code relationship allows a promo code to be removed while the rentals that used it remain, because the foreign key is nullable. Where history matters, setting a status such as `INACTIVE` is preferable to physical deletion.

3.7 Constraint Verification

The integrity rules of the schema can be checked by attempting statements that violate them. Table 3.1 lists the checks together with the error that MySQL is expected to return. Destructive or invalid statements should be run against a test copy of the database.

Table 3.1: Constraint Verification Cases

ID	Statement attempted	Expected outcome
T1	Insert a user whose email already exists	Rejected: error 1062, duplicate entry
T2	Insert a vehicle with an existing registration number	Rejected: error 1062, duplicate entry
T3	Insert a vehicle with a negative `daily_rate`	Rejected: error 3819, check constraint violated
T4	Insert a rental whose return time is not after pickup	Rejected: error 3819, check constraint violated
T5	Insert a payment with `amount` equal to 0	Rejected: error 3819, check constraint violated
T6	Insert a rental that refers to a non-existent user	Rejected: error 1452, foreign-key failure
T7	Delete a vehicle that has rentals	Rejected: error 1451, row is referenced
T8	Insert a rental with `promo_id` set to `NULL`	Accepted, since the promo reference is optional
T9	Insert two payments without provider identifiers	Both accepted, since unique indexes allow multiple `NULL` values

Chapter 4

Conclusion

4.1 Summary of Project Work

This project covered the design and implementation of the relational database of the Vehicle Rental Management System. The work included modelling the rental domain with an entity-relationship diagram, mapping it to seven connected MySQL tables, defining keys and constraints, creating indexes for common access paths, and writing SQL for retrieval, reporting and transactional booking.

4.2 Achievement of Project Objectives

The project objectives were met at the level of database design. The schema models users, vehicles, images, promotional codes, rentals, payments and maintenance with primary keys, seven foreign keys, six unique constraints, seventeen check constraints and twelve secondary indexes. Normalisation was applied, with three documented exceptions (Table 2.11): the postal-code dependency in the users table, the stored price snapshot in rentals, and the user reference stored in payments. The queries and the booking transaction show how the schema supports the operations of a rental business, and the constraint-verification cases (Table 3.1) give a repeatable way to check the integrity rules.

4.3 Lessons Learned

The main lesson is that the database is the foundation of the system's correctness and not merely a store of data. Choosing keys, referential actions and constraints at the design stage prevents whole classes of errors, such as orphan records and negative amounts, that application code would otherwise have to catch. The project also showed that some rules cannot be expressed by constraints alone. Preventing overlapping bookings, for example, needs locking or procedural logic, and keeping stored totals consistent with their components needs discipline in the application. Finally, retaining the agreed price in each rental showed that a small, deliberate departure from strict normalisation can be the correct design when history must be preserved.

Chapter 5

Future Work

5.1 Suggestions for Future Enhancements

- **Prevent overlapping bookings in the database:** add a stored procedure or a `BEFORE INSERT` trigger on `rentals` that performs the overlap check of Listing 3.14, together with row locking, so the rule holds for every client and not only for the application.
- **Tie payments to the rental owner:** add a unique key on `(rental_id, user_id)` in `rentals` and a composite foreign key from `payments`, so that the database guarantees that a payment's user owns the rental.
- **Strengthen promotion rules:** add check constraints for `maximum_discount >= 0` and `used_count <= usage_limit`, and update `used_count` inside the booking transaction.
- **Reporting views and procedures:** define views such as available vehicles and rental summaries, and stored procedures for booking and cancellation, to simplify application queries.
- **Address normalisation:** move city and state into a postal-code lookup table if strict third normal form is required for the users table.
- **Index review:** examine the query plans produced by `EXPLAIN` on representative data to confirm that each index is useful, and remove those that are not.
- **Backup and recovery:** introduce a regular backup schedule and periodically test restoring from it.

5.2 Areas for Further Research

Future work could study the performance of the schema at larger scale, including partitioning of the rentals table by date and read replication for reporting. Other directions include forecasting vehicle demand to support dynamic pricing, predicting maintenance needs from the maintenance history, and extending the payment model to support several payment providers and partial refunds.

References

[1] Oracle Corporation. *MySQL 8.4 Reference Manual*. Available at: `https://dev.mysql.com/doc/refman/8.4/en/`

[2] Oracle Corporation. *MySQL Reference Manual: CREATE TABLE Statement*. Available at: `https://dev.mysql.com/doc/refman/8.4/en/create-table.html`

[3] Oracle Corporation. *MySQL Reference Manual: InnoDB and Foreign Key Constraints*. Available at: `https://dev.mysql.com/doc/refman/8.4/en/innodb-foreign-key-constraints.html`

[4] Oracle Corporation. *MySQL Reference Manual: Optimization and Indexes*. Available at: `https://dev.mysql.com/doc/refman/8.4/en/optimization-indexes.html`

[5] A. Silberschatz, H. F. Korth and S. Sudarshan. *Database System Concepts*, 7th ed. McGraw-Hill Education, 2019.

[6] R. Elmasri and S. B. Navathe. *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.